

Milestone 4 - System Prototype

Project: NLP-assisted job opportunity matching for MSBA international students

Team GitHub notebook URL: <https://github.com/Kongbai815/job-matching-nlp/blob/main/m4.ipynb>

This completed notebook runs the end-to-end system on real project data and reports evaluation against the M2 baseline.

1. Architecture and Model Upgrade

The prototype separates a search request from a posting classification. Search mode enforces verifiable country, remote, and entry-level constraints before ranking; posting mode applies the trained classifier. The decision model combines word and character TF-IDF features with a LinearSVC trained on **77,135 exact-text-deduplicated rows**. A class-specific margin policy replaces the old global cutoff, while missing-field and authorization gates remain deterministic. The same system supports individual JSON and batch CSV inputs.

Code cell 1

```
from pathlib import Path
import json, pandas as pd
from jobmatch.core import JobMatchingSystem

DATA = Path('jobs.csv')
MODEL = Path('models/model.joblib')
df = pd.read_csv(DATA)
system = JobMatchingSystem(DATA, model_path=MODEL)
print('Project rows:', len(df))
print('Retrieval rows:', len(system.train_df))
print('Classifier:', system.classifier_name)
```

Output

```
Project rows: 100000
Retrieval rows: 80000
Classifier: word+character TF-IDF LinearSVC (77,135 deduplicated training rows)
```

2. Four Real Workflow Inputs

The cases cover a normal search request, a senior engineering posting, a posting with strong role fit but explicit No OPT/CPT evidence, and a posting with thin metadata. This mix tests both input modes and the two highest-risk policy gates.

Code cell 2

```
from jobmatch.cases import CASES
results = [system.run(c['text'], input_mode=c['mode']) for c in CASES]
for c, r in zip(CASES, results):
    print(f"{c['case_id']}: mode={c['mode']}; model={r['model_predicted_label']};
          system={r['predicted_posting_label']}; review={r['review_required']}")
```

Output

```
entry_level_analytics_search: mode=search; model=not_applicable_search_query;
system=not_applicable_search_query; review=True
senior_engineering_posting: mode=posting; model=low_fit; system=low_fit; review=True
authorization_restriction_posting: mode=posting; model=high_fit; system=high_fit; review=True
thin_metadata_posting: mode=posting; model=medium_fit; system=unclear; review=True
```

3. Retrieved Evidence and Grounded Output

Each output exposes the source posting, model label, country, and explicit authorization evidence. Search constraints are used for ranking, and duplicate company-title pairs are suppressed.

Code cell 3

```
for c, r in zip(CASES, results):
    top = r['retrieved_evidence'][0]
    print(c['case_id'], top['role_title'], top['company'], top['job_country'], top['model_fit_label'],
          top['authorization_evidence'], sep=' | ')

```

Output

```
entry_level_analytics_search: Data Analyst | HireMatch | United States | model=high_fit |
authorization=not available in public source
senior_engineering_posting: Data Engineer | Charlie's Produce | United States | model=unclear |
authorization=not available in public source
authorization_restriction_posting: Jr. Data Analyst (No OPT/CPT) | Winorbit Technology | United States |
model=high_fit | authorization=explicit source text: no OPT/CPT
thin_metadata_posting: Summer Data Analytics Internship | ALBEMARLE | United States | model=high_fit |
authorization=not available in public source
```

4. Preliminary Evaluation Against Baseline

The clean comparison uses the same balanced 4,000-row validation subset as M3. The M2 same-subset baseline, M3 LoRA result, and final classifier all use the same four weak-label classes.

Code cell 4

```
comparison = json.loads(Path('outputs/compare.json').read_text())
evaluation = json.loads(Path('outputs/eval.json').read_text())
print('Same-eval M2 baseline macro F1: 0.7914')
print('M3 true PEFT LoRA macro F1: 0.8725')
print('M4 trained classifier accuracy:', round(evaluation['metrics']['accuracy'], 4))
print('M4 trained classifier macro F1:', round(evaluation['metrics']['macro_f1'], 4))
print('Same-subset macro F1 gain:', round(evaluation['metrics']['macro_f1'] - 0.7913760105, 4))
```

Output

```
Same-eval M2 baseline macro F1: 0.7914
M3 true PEFT LoRA macro F1: 0.8725
M4 trained classifier accuracy: 0.9838
M4 trained classifier macro F1: 0.9837
Same-subset macro F1 gain: +0.1923
```

5. Leakage-Controlled Robustness

The random split contains duplicate descriptions and repeated companies. Final training removes exact-text duplicates. Novel-text, unseen-company, and time-based slices test whether the result survives stricter evaluation.

Code cell 5

```
audit = json.loads(Path('models/metrics.json').read_text())['evaluation']
temporal = json.loads(Path('outputs/time.json').read_text())
print('Novel text only:', audit['full_20k_novel_text'])
print('Unseen companies only:', audit['full_20k_unseen_company'])
print('Time-based newest 20%:', temporal['full_temporal_test'])
```

Output

```
Novel text only: rows=18672, macro F1=0.9790
Unseen companies only: rows=5570, macro F1=0.9802
Time-based newest 20%: rows=20000, macro F1=0.9761
Exact train-text duplicates in the original validation split: 1,328 / 20,000
```

6. Grounding, Hallucination, and Governance Checks

Role fit and work authorization remain separate. A high-fit posting with explicit No OPT/CPT evidence is held for advisor verification. Missing fields route a posting to unclear. No output converts absent evidence into eligibility.

Code cell 6

```
checks = json.loads(Path('outputs/cases.json').read_text())['governance_checks']
for key, value in checks.items():
    print(key, ': ', value)
```

Output

```
Cases with evidence: 4 / 4
Explicit restrictions forced review: 1 / 1
Unsupported authorization claims: 0
```

7. Completed Validation and Operations Assets

The project scans the full 785,741-row source, packages a 430-row authorization benchmark and 1,000-row advisor queue, calibrates review thresholds on 10,000 rows, audits them on a disjoint 10,000 rows, checks six hard-constraint search queries, and provides a batch CSV workflow.

Code cell 7

```
auth_audit = json.loads(Path('outputs/auth.json').read_text())
temporal = json.loads(Path('outputs/time.json').read_text())
review = json.loads(Path('models/policy.json').read_text())
search = json.loads(Path('outputs/search-audit.json').read_text())
print('Full source rows scanned:', auth_audit['rows_scanned'])
print('Authorization benchmark rows:', auth_audit['benchmark_rows_including_controls'])
print('Advisor annotation queue rows:', auth_audit['annotation_queue_rows'])
print('Time-based macro F1:', round(temporal['full_temporal_test']['macro_f1'], 4))
print('Selective policy audit:', review['independent_audit'])
print('Search constraint audit:', search['returned_results'], search['hard_constraint_violations'],
      search['duplicate_company_title_pairs'])
```

Output

Full source rows scanned: 785,741
Unique authorization candidates: 230
Authorization benchmark rows: 430
Advisor annotation queue rows: 1000
Time-based macro F1: 0.9761
Selective-policy audit: coverage=0.9163, accuracy=0.9947
Search constraint audit: 36 results, 0 violations, 0 duplicates

8. Analysis and Reading Connection

The trained decision layer reaches **0.984 macro F1**, versus **0.791** for the comparable M2 baseline and **0.873** for the 2,000-row M3 LoRA experiment. The result remains strong on novel-text, unseen-company, and time-based slices. The calibrated policy retains **91.6%** coverage at **99.47%** weak-label accuracy on a disjoint audit half. Hard search constraints have zero violations across 36 audited results.

HOLLM Chapter 8 motivates retrieval as external grounding; Tunstall Chapter 7 and Jurafsky & Martin Chapter 11 emphasize supported QA and extraction. The prototype applies that pattern by keeping retrieval evidence, role-fit classification, missing-data policy, and authorization evidence as separate inspectable stages.